

```

1 import json
2
3 # Construção manual da estrutura do Jupyter Notebook JSON
4 notebook = {
5     "cells": [
6         {
7             "cell_type": "markdown",
8             "metadata": {},
9             "source": [
10                 "# Exercícios de Estruturas de Dados em Python: Nós, Lista Encadeada e Fila de Atendimento\n",
11                 "\n",
12                 "Este notebook contém as respostas explicadas e executáveis de todos os exercícios propostos sobre pont
13             ]
14         },
15         {
16             "cell_type": "markdown",
17             "metadata": {},
18             "source": [
19                 "---\n",
20                 "## Exercício 1 – Referências\n",
21                 "\n",
22                 "### Código:\n",
23                 ```python\n",
24                 a = [10, 20, 30]\n",
25                 b = a\n",
26                 b.append(40)\n",
27                 print("a:", a)\n",
28                 print("b:", b)\n",
29                 print("id(a):", id(a))\n",
30                 print("id(b):", id(b))\n",
31                 ```\n",
32                 "\n",
33                 "### Respostas:\n",
34                 "1. **Qual será a saída?**\n",
35                 "    - `a: [10, 20, 30, 40]`\n",
36                 "    - `b: [10, 20, 30, 40]`\n",
37                 "    - `id(a)` e `id(b)` exibirão exatamente o mesmo número de identificação na memória.\n",
38                 "2. **`a` e `b` representam o mesmo objeto?**\n",
39                 "    - **Sim.** Ambas as variáveis referenciam a mesma lista alocada na memória RAM.\n",
40                 "3. **Por quê?**\n",
41                 "    - Em Python, a instrução `b = a` não cria uma cópia dos dados. Ela apenas cria uma nova referência
42                 "4. **O que aconteceria com `b = a.copy()`?\n",
43                 "    - Seria criada uma **cópia rasa (shallow copy)** em uma nova região de memória. Modificações em `b`
44             ]
45         },
46         {
47             "cell_type": "code",
48             "execution_count": None,
49             "metadata": {},
50             "outputs": [],
51             "source": [
52                 "# Demonstração do Exercício 1\n",
53                 a = [10, 20, 30]\n",
54                 b = a\n",
55                 b.append(40)\n",
56                 "\n",
57                 print("a:", a)\n",
58                 print("b:", b)\n",
59                 print("id(a):", id(a))\n",
60                 print("id(b):", id(b))\n",
61                 print("Mesmo objeto em memória?", id(a) == id(b))\n",
62                 "\n",
63                 "# Teste com .copy()\n",
64                 c = a.copy()\n",
65                 c.append(50)\n",
66                 print("\nApós c = a.copy() e c.append(50):")\n",
67                 print("a:", a)\n",
68                 print("c:", c)\n",
69                 print("id(a) == id(c):", id(a) == id(c))
70             ]
71         },
72         {
73             "cell_type": "markdown",
74             "metadata": {},
75             "source": [
76                 "---\n",
77                 "## Exercício 2 – Construindo uma cadeia\n",
78                 "\n",
79                 "### Objetivo:\n",
80                 "Criar três nós (`n1`, `n2`, `n3`), encadeá-los no formato `A -> B -> C -> None` e percorrer a estrutura
81             ]
82         }
83     ]
84 }

```

```

82     },
83     {
84         "cell_type": "code",
85         "execution_count": None,
86         "metadata": {},
87         "outputs": [],
88         "source": [
89             "class Node:\n",
90             "    def __init__(self, valor):\n",
91             "        self.valor = valor\n",
92             "        self.proximo = None\n",
93             "\n",
94             "# Criando os nós\n",
95             "n1 = Node(\"A\")\n",
96             "n2 = Node(\"B\")\n",
97             "n3 = Node(\"C\")\n",
98             "\n",
99             "# Conectando: A -> B -> C -> None\n",
100            "n1.proximo = n2\n",
101            "n2.proximo = n3\n",
102            "\n",
103            "# Percorrendo a estrutura\n",
104            "atual = n1\n",
105            "while atual is not None:\n",
106            "    print(atual.valor)\n",
107            "    atual = atual.proximo"
108        ]
109    },
110    {
111        "cell_type": "markdown",
112        "metadata": {},
113        "source": [
114            "---\n",
115            "## Exercício 3 – Depuração\n",
116            "\n",
117            "### Análise do Código com Erro:\n",
118            "```python\n",
119            "n1 = Node(10)\n",
120            "n2 = Node(20)\n",
121            "n3 = Node(30)\n",
122            "n1.proximo = n2\n",
123            "n2.proximo = n3\n",
124            "atual = n1\n",
125            "while atual is not None:\n",
126            "    print(atual.valor)\n",
127            "```\n",
128            "\n",
129            "### Respostas:\n",
130            "1. **Qual é o problema?**\n",
131            "   - O laço `while` não atualiza o ponteiro de navegação (`atual`).\n",
132            "2. **Por que o programa não termina?**\n",
133            "   - A variável `atual` permanece eternamente apontando para `n1`. A condição `atual is not None` nunca será falsa.\n",
134            "3. **Qual linha deve ser acrescentada?**\n",
135            "   - `atual = atual.proximo` dentro do bloco do `while`.\n",
136            "4. **Qual será a saída depois da correção?**\n",
137            "   - `10`\n",
138            "   - `20`\n",
139            "   - `30`"
140        ]
141    },
142    {
143        "cell_type": "code",
144        "execution_count": None,
145        "metadata": {},
146        "outputs": [],
147        "source": [
148            "# Código corrigido do Exercício 3\n",
149            "class Node:\n",
150            "    def __init__(self, valor):\n",
151            "        self.valor = valor\n",
152            "        self.proximo = None\n",
153            "\n",
154            "n1 = Node(10)\n",
155            "n2 = Node(20)\n",
156            "n3 = Node(30)\n",
157            "n1.proximo = n2\n",
158            "n2.proximo = n3\n",
159            "\n",
160            "atual = n1\n",
161            "while atual is not None:\n",
162            "    print(atual.valor)\n",
163            "    atual = atual.proximo # Linha acrescentada para avançar o ponteiro"

```

```

164     ]
165 },
166 {
167     "cell_type": "markdown",
168     "metadata": {},
169     "source": [
170         "---\n",
171         "## Exercícios 18 a 23 – Sistema de Atendimento de uma Clínica (Com Desafio Prioritário)\n",
172         "\n",
173         "### Descrição do Sistema:\n",
174         "O sistema gerencia uma fila de atendimento para uma clínica médica usando uma lista encadeada dinâmica\n",
175         "\n",
176         "### Estrutura Implementada:\n",
177         "1. `Paciente`: Armazena nome, idade e prioridade (`Normal` ou `Prioridade`).\n",
178         "2. `Node`: Encapsula o paciente e o ponteiro `proximo`.\n",
179         "3. `FilaAtendimento`: Controla os ponteiros `inicio` e `fim`, fornecendo métodos para:\n",
180         "    - `adicionar(paciente)`: Insere respeitando a prioridade (pacientes prioritários furam a fila de no\n",
181         "    - `atender()`: Remove e retorna o primeiro paciente da fila.\n",
182         "    - `listar()`: Exibe todos os pacientes na fila.\n",
183         "    - `esta_vazia()`: Verifica se há pacientes aguardando.\n",
184         "    - `tamanho()`: Retorna o número atual de pacientes na fila."
185     ]
186 },
187 {
188     "cell_type": "code",
189     "execution_count": None,
190     "metadata": {},
191     "outputs": [],
192     "source": [
193         "class Paciente:\n",
194         "    def __init__(self, nome, idade, prioridade):\n",
195         "        self.nome = nome\n",
196         "        self.idade = idade\n",
197         "        self.prioridade = prioridade # `Normal` ou `Prioridade`\n",
198         "\n",
199         "    def __repr__(self):\n",
200         "        return f"{self.nome} ({self.idade} anos) [{self.prioridade}]\n",
201         "\n",
202         "class Node:\n",
203         "    def __init__(self, paciente):\n",
204         "        self.paciente = paciente\n",
205         "        self.proximo = None\n",
206         "\n",
207         "class FilaAtendimento:\n",
208         "    def __init__(self):\n",
209         "        self.inicio = None\n",
210         "        self.fim = None\n",
211         "        self._tamanho = 0\n",
212         "\n",
213         "    def esta_vazia(self):\n",
214         "        return self.inicio is None\n",
215         "\n",
216         "    def tamanho(self):\n",
217         "        return self._tamanho\n",
218         "\n",
219         "    def adicionar(self, paciente):\n",
220         "        novo_no = Node(paciente)\n",
221         "        self._tamanho += 1\n",
222         "\n",
223         "        # Caso 1: Fila Vazia\n",
224         "        if self.esta_vazia():\n",
225         "            self.inicio = novo_no\n",
226         "            self.fim = novo_no\n",
227         "            return\n",
228         "\n",
229         "        # Caso 2: Atendimento Prioritário (Desafio 23)\n",
230         "        if paciente.prioridade == "Prioridade":\n",
231         "            # Se o primeiro da fila for Normal, o prioritário assume a cabeça\n",
232         "            if self.inicio.paciente.prioridade == "Normal":\n",
233         "                novo_no.proximo = self.inicio\n",
234         "                self.inicio = novo_no\n",
235         "            else:\n",
236         "                # Percorre até achar o último nó com prioridade\n",
237         "                atual = self.inicio\n",
238         "                while atual.proximo and atual.proximo.paciente.prioridade == "Prioridade":\n",
239         "                    atual = atual.proximo\n",
240         "                \n",
241         "                novo_no.proximo = atual.proximo\n",
242         "                atual.proximo = novo_no\n",
243         "                if novo_no.proximo is None:\n",
244         "                    self.fim = novo_no

```

```

247         "        else:\n",
248         "        # Caso 3: Paciente Normal vai para o final da fila\n",
249         "        self.fim.proximo = novo_no\n",
250         "        self.fim = novo_no\n",
251         "\n",
252         "        def atender(self):\n",
253         "        if self.esta_vazia():\n",
254         "        print(\"Nenhum paciente para atender.\")\n",
255         "        return None\n",
256         "\n",
257         "        paciente_atendido = self.inicio.paciente\n",
258         "        self.inicio = self.inicio.proximo\n",
259         "        self._tamanho -= 1\n",
260         "\n",
261         "        if self.inicio is None:\n",
262         "        self.fim = None\n",
263         "\n",
264         "        return paciente_atendido\n",
265         "\n",
266         "        def listar(self):\n",
267         "        if self.esta_vazia():\n",
268         "        print(\"Fila vazia.\")\n",
269         "        return\n",
270         "\n",
271         "        atual = self.inicio\n",
272         "        i = 1\n",
273         "        while atual:\n",
274         "        print(f\"{i}. {atual.paciente}\")\n",
275         "        atual = atual.proximo\n",
276         "        i += 1\n",
277         "\n",
278         "\n",
279         "# --- Teste Completo (Exercícios 22, 23 e 25) ---\n",
280         "fila = FilaAtendimento()\n",
281         "\n",
282         "# Adicionando pacientes de teste\n",
283         "fila.adicionar(Paciente(\"Ana\", 32, \"Normal\"))\n",
284         "fila.adicionar(Paciente(\"Bruno\", 70, \"Normal\"))\n",
285         "fila.adicionar(Paciente(\"Carlos\", 45, \"Prioridade\"))\n",
286         "fila.adicionar(Paciente(\"Daniela\", 28, \"Normal\"))\n",
287         "fila.adicionar(Paciente(\"Eduardo\", 68, \"Prioridade\"))\n",
288         "\n",
289         "print(\"--- Fila Inicial Organizada ---\")\n",
290         "fila.listar()\n",
291         "print(f\"Total na fila: {fila.tamanho()} paciente(s)\")\n",
292         "\n",
293         "print(\"\\n--- Atendimentos ---\")\n",
294         "p1 = fila.atender()\n",
295         "print(\"Atendendo:\", p1)\n",
296         "p2 = fila.atender()\n",
297         "print(\"Atendendo:\", p2)\n",
298         "\n",
299         "print(\"\\n--- Fila Após Atendimentos ---\")\n",
300         "fila.listar()\n",
301         "print(f\"Total restante: {fila.tamanho()} paciente(s)\")\n",
302     ]\n",
303 },\n",
304 {\n",
305     "cell_type": "markdown",\n",
306     "metadata": {},\n",
307     "source": [\n",
308         "        \"---\",\n",
309         "        \"### Exercícios 26 e 27 – Checklist de Aprendizagem e Reflexão Final\",\n",
310         "        \"\\n\",\n",
311         "        \"### Reflexão Final (Respostas do Questionário):\",\n",
312         "        \"\\n\",\n",
313         "        \"1. **O que é uma referência em Python?**\",\n",
314         "        \"> Uma referência é um valor (endereço de memória) que indica onde um objeto real está armazenado na memória.\",\n",
315         "        \"\\n\",\n",
316         "        \"2. **Qual a diferença entre `b = a` e `b = a.copy()`?\",\n",
317         "        \"> `b = a` cria um novo apelido apontando para a mesma região de memória; alterar `b` modifica `a`.\", \n",
318         "        \"\\n\",\n",
319         "        \"3. **Por que uma estrutura encadeada precisa de referências?\",\n",
320         "        \"> Porque os elementos (nós) de uma estrutura dinâmica não ficam armazenados em posições contíguas na memória.\",\n",
321         "        \"\\n\",\n",
322         "        \"4. **Qual erro você encontrou durante a depuração?\",\n",
323         "        \"> O erro do loop infinito causado pela ausência da instrução `atual = atual.proximo` ao percorrer a fila.\",\n",
324         "        \"\\n\",\n",
325         "        \"5. **Como referências ajudam a compreender estruturas dinâmicas?\",\n",
326         "        \"> Elas mostram como construir coleções de dados flexíveis sem limite fixo de tamanho, realizando inserções e remoções eficientes.\",\n",
327     ]\n",
328 }

```

```
330     "metadata": {  
331         "language_info": {  
332             "name": "python"  
333         }  
334     },  
335     "nbformat": 4,  
336     "nbformat_minor": 2  
337 }  
338  
339 output_filename = "exercicios_lista_encadeada.ipynb"  
340 with open(output_filename, "w", encoding="utf-8") as f:  
341     json.dump(notebook, f, indent=2, ensure_ascii=False)  
342  
343 print(f"Arquivo '{output_filename}' gerado com sucesso!")
```